

Tema 2: Punteros, Arrays y Memoria Dinamica

Este es probablemente el tema mas preguntado en los exámenes de Sistemas Operativos. Alrededor del treinta por ciento de las preguntas tienen que ver con punteros, arrays, malloc, free y sizeof. Presta mucha atencion.

Que es un puntero

Un puntero es una variable que contiene una direccion de memoria. Se declara poniendo un asterisco delante del nombre: por ejemplo, `int asterisco ptr` declara un puntero a entero. Hay dos operadores fundamentales para trabajar con punteros. El operador ampersand, llamado address-of, obtiene la direccion de una variable. El operador asterisco, llamado dereference o indireccion, accede al valor que esta en la direccion apuntada por el puntero.

Por ejemplo, si tienes `int x` igual a diez e `int asterisco ptr`, y luego haces `ptr igual ampersand x`, ahora `ptr` apunta a `x`. Si haces `asterisco ptr igual a veinte`, estas cambiando el valor de `x` a traves del puntero.

Una regla importantisima: un puntero que no apunta a nada es peligroso. Siempre inicializa los punteros a NULL si todavia no apuntan a nada, y comprueba que no sean NULL antes de usarlos. Acceder a un puntero sin inicializar provoca un segmentation fault, que es una violacion de segmentacion.

Aritmetica de punteros

Los punteros pueden sumarse y restarse, pero las operaciones se hacen en multiples del tamaño del tipo apuntado. Si tienes un puntero a `char` y le sumas cuatro, avanza cuatro bytes, porque un `char` ocupa un byte. Pero si tienes un puntero a `int` y le sumas cuatro, avanza dieciseis bytes, porque un `int` ocupa cuatro bytes y cuatro por cuatro son dieciseis.

Esto es clave para entender las preguntas de examen sobre punteros y arrays. Vamos con un ejemplo tipico: tienes un array `arr` de diez enteros, inicializado del cero al nueve. Haces `p igual a arr mas dos`, así que `p` apunta a `arr` sub dos, que vale dos. Luego haces `p igual a ampersand de p sub uno`. Aqui `p sub uno` es lo mismo que `asterisco de p mas uno`, que es `arr sub tres`, que vale tres. Y `ampersand de eso` te da la direccion de `arr sub tres`. Así que ahora `p` apunta a `arr sub tres`. Finalmente, `p sub dos` es lo mismo que `arr sub cinco`, que vale cinco. La respuesta es cinco. Este tipo de ejercicio es clasico en los tests.

Arrays en C

Un array es una secuencia contigua de elementos del mismo tipo. Se declara con corchetes: `int lista de N` reserva memoria para `N` enteros. Los indices van de cero a `N` menos uno, y `C` no comprueba los limites. Si accedes fuera del rango, no te da error de compilacion, simplemente accedes a memoria que no te pertenece, lo que puede causar un segmentation fault o resultados impredecibles.

Un array es basicamente azucar sintactico para punteros. Cuando escribes `lista sub i`, el

compilador lo convierte en asterisco de lista mas i. El nombre del array sin corchetes es un puntero al primer elemento.

Aqui viene la trampa del sizeof, que es pregunta de examen segura. Si aplicas sizeof a un array declarado en la pila, obtienes el tamaño total en bytes. Por ejemplo, int p de cinco te da sizeof p igual a veinte, porque son cinco enteros de cuatro bytes cada uno. Pero cuidado: si el array es un parametro de funcion, ya no es un array, es un puntero, y sizeof te dara el tamaño del puntero, que son ocho bytes en arquitectura de sesenta y cuatro bits. Esto se pregunta constantemente.

Otro ejemplo clasico: si tienes un array de cinco enteros y haces un for de cero hasta sizeof p, estas iterando veinte veces en vez de cinco, porque sizeof p en ese contexto te da veinte bytes, no cinco elementos. Esto provocaria acceder a posiciones fuera del array e imprimir basura.

Cadenas de caracteres

Una cadena de caracteres, o string, es un array de chars terminado con el caracter nulo, que se escribe como barra invertida cero. Si declaras char str igual a comilla hola comilla, el compilador reserva cinco bytes: h, o, l, a, y el caracter nulo. El caracter nulo es lo que marca el final de la cadena, y todas las funciones de string como strlen, strcmp, strcpy y strcat dependen de el.

Si modificas un caracter de la cadena y pones un cero en medio, por ejemplo p sub dos igual a cero, la cadena se acorta hasta ese punto. strlen te devolvera dos en vez de cuatro, porque strlen cuenta hasta encontrar el caracter nulo.

Las funciones de string mas importantes son: strlen, que devuelve la longitud sin contar el nulo; strcmp, que compara dos strings y devuelve cero si son iguales; strcpy, que copia una string a otra; strcat, que concatena; y sprintf, que es como printf pero escribe en una cadena en vez de en pantalla.

Estructuras

Una estructura es un tipo de dato compuesto que agrupa variables de distintos tipos. Se declara con struct seguido del nombre y los campos entre llaves. Para crear un tipo mas comodo, usamos typedef. El operador punto accede a campos de una estructura directa, y el operador flecha, que se escribe como menos mayor que, accede a campos a traves de un puntero a estructura. El operador flecha es equivalente a asterisco p punto campo. Esto es pregunta de examen: el operador flecha atraviesa un puntero y accede al campo de un registro.

El tamaño de una estructura no es necesariamente la suma de sus campos, porque el compilador puede insertar relleno para alinear los datos en memoria.

Memoria dinamica: malloc y free

Para reservar memoria en tiempo de ejecucion usamos malloc, que recibe el numero de bytes a reservar y devuelve un puntero void a la memoria reservada. Si no hay memoria suficiente,

devuelve NULL. La memoria reservada puede contener cualquier valor basura; no se inicializa a cero.

Para liberar memoria usamos free, que recibe el puntero devuelto por malloc. Hay reglas importantísimas: solo puedes liberar memoria que vino de malloc; debes liberar la memoria cuando ya no la necesitas; y no debes usar un puntero después de haber liberado su memoria.

Los errores más comunes con memoria dinámica, que se preguntan constantemente en exámenes, son los siguientes.

Primero, el memory leak o fuga de memoria. Ocurre cuando reservas memoria con malloc y luego pierdes el puntero sin haber llamado a free. Un ejemplo clásico: haces p igual a malloc, y luego inmediatamente haces p igual a otra cosa, como el inicio de una lista enlazada. La memoria que reservaste con malloc se pierde para siempre.

Segundo, retornar un puntero a una variable local. Si una función declara una variable local y retorna un puntero a ella, ese puntero queda inválido al salir de la función, porque la variable se destruye. Acceder a ese puntero provoca un segmentation fault o comportamiento indefinido. La solución es usar malloc dentro de la función para que la memoria persista.

Tercero, el sizeof incorrecto. Si haces malloc de cien por sizeof de char asterisco en vez de sizeof de char, estás reservando probablemente ochocientos bytes en vez de cien, porque sizeof de char asterisco es ocho bytes en sistemas de sesenta y cuatro bits. El programa puede que funcione, pero estarías desperdiciando memoria.

Cuarto, usar memset con sizeof de un puntero en vez del tamaño real del bloque. Si haces memset de p, cero, sizeof de p, solo inicializas ocho bytes, los del puntero, en vez de todo el bloque de memoria. El resto queda sin inicializar.

El operador sizeof: la trampa definitiva

Sizeof aplicado a un puntero devuelve el tamaño del puntero, que es cuatro bytes en treinta y dos bits y ocho bytes en sesenta y cuatro bits. No te dice cuánto ocupa el bloque de memoria al que apunta. El compilador no puede saber eso porque malloc se resuelve en tiempo de ejecución.

Sizeof aplicado a un array en la pila sí devuelve el tamaño total. Pero si pasas ese array como parámetro a una función, se degrada a puntero y sizeof dará el tamaño del puntero.

Pregunta típica: tienes char asterisco array igual a malloc de cierto tamaño. Haces printf de sizeof de array. La respuesta es ocho en sesenta y cuatro bits, no el tamaño del malloc. Porque array es un puntero y sizeof devuelve el tamaño del puntero.

Paso por valor y paso por referencia

En C, todos los argumentos se pasan por valor, es decir, se pasa una copia. Si quieres que una función modifique una variable del llamador, debes pasar un puntero a esa variable. Dentro de la función, modificas el valor usando el operador asterisco sobre el puntero. Esto es lo que se llama paso por referencia simulado.

Por ejemplo, si quieres una funcion que cambie el valor de un entero, el parametro debe ser `int *` y dentro de la funcion haces `*ptr` igual al nuevo valor. Al llamar a la funcion, pasas `&ptr` de la variable.